

Engenharia de Software – PUC Campinas

Algoritmos e Linguagens de Programação

Prof. Dr. Paulo Cesar de Alvarenga Lucci

Matrizes

Matrizes

- O QUE SÃO?

- Matrizes são vetores de duas (ou mais) dimensões.
 - Matrizes podem ter mais de 2 dimensões, mas o uso prático comum é 2D.
- Usadas para representar tabelas, grades, mapas, etc.
- Cada elemento é acessado por dois índices (quando 2D): linha e coluna.
 - Em C, os índices começam em 0.
- Podem armazenar tipos primitivos, structs, vetores (matrizes), ponteiros e enums.
- A alocação de memória pode ser feita de forma estática ou dinâmica.
- Sintaxe da alocação estática de matriz 2D:

tipo_armazenado nome_da_matriz[qtd_de_linhas][qtd_de_colunas].

Matrizes

○ EXEMPLO:

```
#include <stdio.h>

int main()
{
    // Declaração de uma matriz 3x4
    //(3 linhas e 4 colunas, ou seja, 12 elementos)
    int matriz[3][4];
    int i, j;

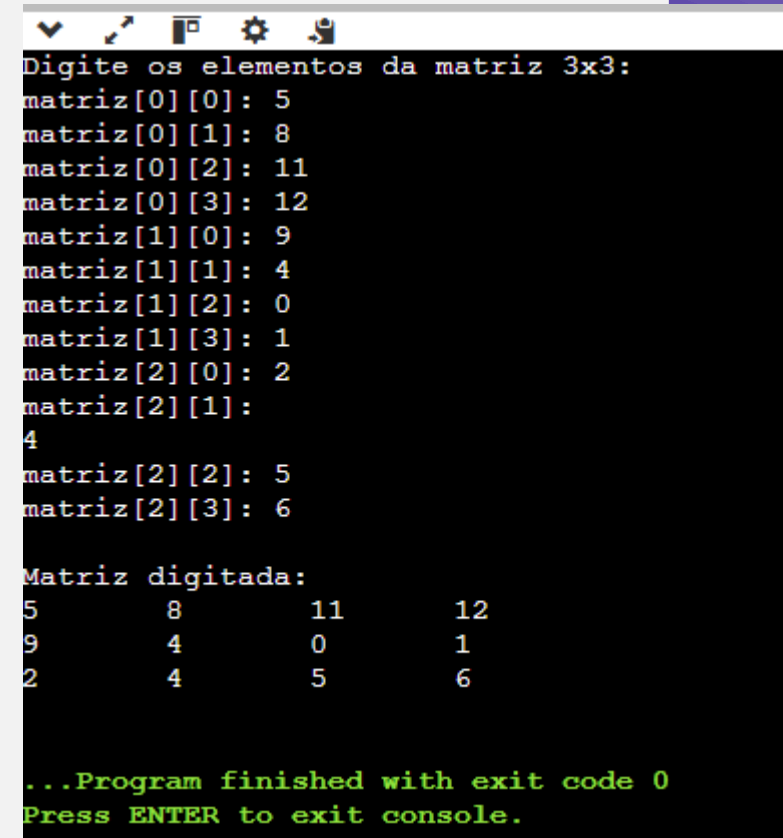
    // Leitura dos elementos da matriz
    printf("Digite os elementos da matriz 3x3:\n");
    for (i = 0; i < 3; i++)
    {
        for (j = 0; j < 4; j++)
        {
            printf("matriz[%d][%d]: ", i, j);
            scanf("%d", &matriz[i][j]);
        }
    }

    // Impressão da matriz
    printf("\nMatriz digitada:\n");
    for (i = 0; i < 3; i++)
    {
        for (j = 0; j < 4; j++)
        {
            printf("%d\t", matriz[i][j]);
        }
        printf("\n");
    }

    return 0;
}
```

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      // Declaração de uma matriz 3x4
6      //(3 linhas e 4 colunas, ou seja, 12 elementos)
7      int matriz[3][4];
8      int i, j;
9
10     // Leitura dos elementos da matriz
11     printf("Digite os elementos da matriz 3x3:\n");
12     for (i = 0; i < 3; i++)
13     {
14         for (j = 0; j < 4; j++)
15         {
16             printf("matriz[%d][%d]: ", i, j);
17             scanf("%d", &matriz[i][j]);
18         }
19     }
20
21     // Impressão da matriz
22     printf("\nMatriz digitada:\n");
23     for (i = 0; i < 3; i++)
24     {
25         for (j = 0; j < 4; j++)
26         {
27             printf("%d\t", matriz[i][j]);
28         }
29         printf("\n");
30     }
31
32     return 0;
33 }
```

Matriz inicializada posteriormente à sua declaração através do teclado.



```

▼ ↗ 📄 ⚙️ 🐞
Digite os elementos da matriz 3x3:
matriz[0][0]: 5
matriz[0][1]: 8
matriz[0][2]: 11
matriz[0][3]: 12
matriz[1][0]: 9
matriz[1][1]: 4
matriz[1][2]: 0
matriz[1][3]: 1
matriz[2][0]: 2
matriz[2][1]:
4
matriz[2][2]: 5
matriz[2][3]: 6

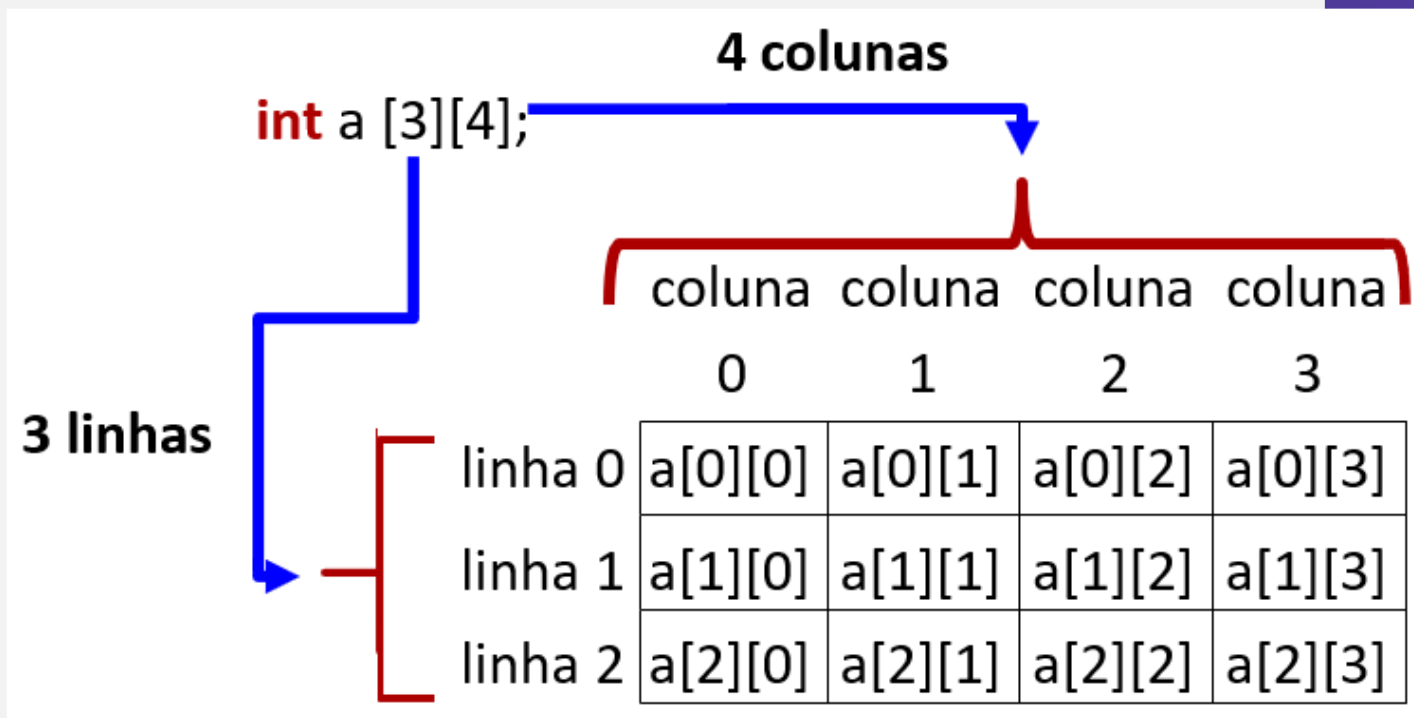
Matriz digitada:
5      8      11      12
9      4      0      1
2      4      5      6

...Program finished with exit code 0
Press ENTER to exit console.
```

Matrizes

- **ATENÇÃO:**

- Nos exemplos apresentados temos matrizes 3x4.
- Pelo fato de a indexação iniciar em 0, os índices das:
 - **Linhas:** vão de 0 a 2 (3 linhas).
 - **Colunas:** de 0 a 3 (4 colunas).



Matrizes

○ EXEMPLO:

- Matrizes inicializadas e preenchidas no código (hardcoded) através de 2 sintaxes.

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int matriz[3][4] = {
6          {10, 20, 30, 40},
7          {50, 60, 70, 80},
8          {90, 100, 110, 120}
9      };
10
11     int i, j;
12
13     // Impressão da matriz
14     printf("Matriz inicializada:\n");
15     for (i = 0; i < 3; i++)
16     {
17         for (j = 0; j < 4; j++)
18         {
19             printf("%d\t", matriz[i][j]);
20         }
21         printf("\n");
22     }
23
24     return 0;
25 }
```

```
main.c
1  #include <stdio.h>
2
3  int main()
4  {
5      int matriz[3][4] = { 10, 20, 30, 40, 50, 60, 70, 80, 90, 100, 110, 120 };
6
7      int i, j;
8
9      // Impressão da matriz
10     printf("Matriz inicializada:\n");
11     for (i = 0; i < 3; i++)
12     {
13         for (j = 0; j < 4; j++)
14         {
15             printf("%d\t", matriz[i][j]);
16         }
17         printf("\n");
18     }
19
20     return 0;
21 }
```

Matrizes

○ ALGUNS ASPECTOS SOBRE MATRIZES:

- Matrizes são chamadas “**quadradas**” quando a quantidade de linhas e colunas forem iguais. Exemplo: `double myMatrix[4][4];`
 - As **matrizes quadradas** têm 2 diagonais:
 - Diagonal principal.
 - Diagonal secundária.
 - A **matriz identidade** é uma matriz quadrada em que na diagonal principal temos 1 e no restante 0.

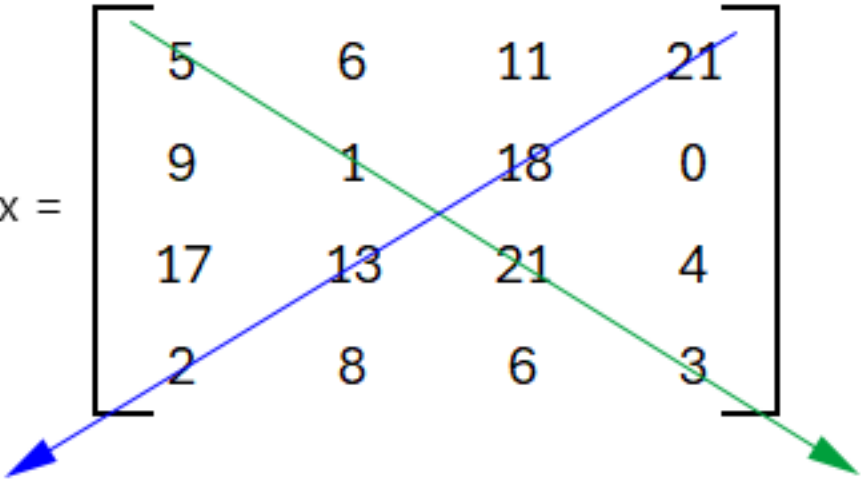
$$\text{myMatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

myMatrix =

$$\begin{bmatrix} 5 & 6 & 11 & 21 \\ 9 & 1 & 18 & 0 \\ 17 & 13 & 21 & 4 \\ 2 & 8 & 6 & 3 \end{bmatrix}$$

Diagonal secundária

Diagonal principal



Matrizes

○ ALGUNS ASPECTOS SOBRE MATRIZES:

- A **matriz transposta** de uma matriz é uma **nova matriz** onde as **linhas viram colunas** e as **colunas viram linhas**, ou seja:
 - Se a matriz original tem tamanho **m x n**, sua transposta terá tamanho **n x m**.
 - Se a matriz original tem elemento **[i][j]**, na transposta esse elemento fica na posição **[j][i]**.

○ Exemplo:

- Matriz original 3x2:

1	2
3	4
5	6

- Matriz transposta 2x3:

1	3	5
2	4	6

```
#include <stdio.h>

int main() {
    // Matriz original A (3x2)
    int A[3][2] = { {1, 2}, {3, 4}, {5, 6} };

    // Matriz transposta At (2x3)
    int At[2][3];

    // Calculando a transposta
    for (int i = 0; i < 3; i++) for (int j = 0; j < 2; j++) At[j][i] = A[i][j];

    // Imprimindo a matriz A
    printf("Matriz A:\n");
    for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 2; j++) printf("%d ", A[i][j]);
        printf("\n");
    }

    // Imprimindo a matriz At
    printf("Matriz transposta At:\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) printf("%d ", At[i][j]);
        printf("\n");
    }

    return 0;
}
```

```
main.c
1 #include <stdio.h>
2
3 int main() {
4     // Matriz original A (3x2)
5     int A[3][2] = { {1, 2}, {3, 4}, {5, 6} };
6
7     // Matriz transposta At (2x3)
8     int At[2][3];
9
10    // Calculando a transposta
11    for (int i = 0; i < 3; i++) for (int j = 0; j < 2; j++) At[j][i] = A[i][j];
12
13    // Imprimindo a matriz A
14    printf("Matriz A:\n");
15    for (int i = 0; i < 3; i++) {
16        for (int j = 0; j < 2; j++) printf("%d ", A[i][j]);
17        printf("\n");
18    }
19
20    // Imprimindo a matriz At
21    printf("Matriz transposta At:\n");
22    for (int i = 0; i < 2; i++) {
23        for (int j = 0; j < 3; j++) printf("%d ", At[i][j]);
24        printf("\n");
25    }
26
27    return 0;
28 }
29
```

Matriz A:

```
1 2
3 4
5 6
```

Matriz transposta At:

```
1 3 5
2 4 6
```

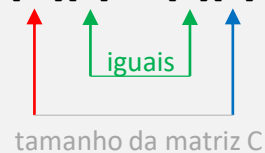
Matrizes

- ALGUNS ASPECTOS SOBRE MATRIZES:

- Multiplicação de matrizes:

- Para que seja possível multiplicar duas matrizes (A x B), é obrigatório que:

- $A[m][n] \times B[n][o] = C[m][o]$ ou seja:



- A quantidade de colunas da matriz A deve ser igual à quantidade de linhas da matriz B.
 - A matriz resultante desse produto (matriz C) terá tamanho $m \times o$.

- Como o algoritmo funciona?

- Em poucas palavras:

- Para preencher a posição $C[i][j]$, percorremos a linha i da matriz A e a coluna j da matriz B, multiplicando os elementos e somando os produtos.

Matrizes

○ Exemplo:

$$A = \begin{bmatrix} 3 & 2 \\ 5 & -1 \end{bmatrix} \quad B = \begin{bmatrix} 6 & 4 & -2 \\ 0 & 7 & 1 \end{bmatrix}$$

Matrizes

○ Exemplo:

$$A = \begin{bmatrix} 3 & 2 \\ 5 & -1 \end{bmatrix} \quad B = \begin{bmatrix} 6 & 4 & -2 \\ 0 & 7 & 1 \end{bmatrix}$$

$$C = \begin{bmatrix} 3 \cdot 6 + 2 \cdot 0 & & \\ & & \\ & & \end{bmatrix}$$

$$C = \begin{bmatrix} 18 & & \\ & & \\ & & \end{bmatrix}$$

$$A = \begin{bmatrix} 3 & 2 \\ 5 & -1 \end{bmatrix} \quad B = \begin{bmatrix} 6 & 4 & -2 \\ 0 & 7 & 1 \end{bmatrix}$$

$$C = \begin{bmatrix} 3 \cdot 6 + 2 \cdot 0 & 3 \cdot 4 + 2 \cdot 7 & 3 \cdot (-2) + 2 \cdot 1 \\ 5 \cdot 6 + (-1) \cdot 0 & & \\ & & \end{bmatrix}$$

$$C = \begin{bmatrix} 18 & 26 & -4 \\ 30 & & \end{bmatrix}$$

$$A = \begin{bmatrix} 3 & 2 \\ 5 & -1 \end{bmatrix} \quad B = \begin{bmatrix} 6 & 4 & -2 \\ 0 & 7 & 1 \end{bmatrix}$$

$$C = \begin{bmatrix} 3 \cdot 6 + 2 \cdot 0 & 3 \cdot 4 + 2 \cdot 7 & \\ & & \\ & & \end{bmatrix}$$

$$C = \begin{bmatrix} 18 & 26 & \\ & & \\ & & \end{bmatrix}$$

$$A = \begin{bmatrix} 3 & 2 \\ 5 & -1 \end{bmatrix} \quad B = \begin{bmatrix} 6 & 4 & -2 \\ 0 & 7 & 1 \end{bmatrix}$$

$$C = \begin{bmatrix} 3 \cdot 6 + 2 \cdot 0 & 3 \cdot 4 + 2 \cdot 7 & 3 \cdot (-2) + 2 \cdot 1 \\ 5 \cdot 6 + (-1) \cdot 0 & 5 \cdot 4 + (-1) \cdot 7 & \\ & & \end{bmatrix}$$

$$C = \begin{bmatrix} 18 & 26 & -4 \\ 30 & 13 & \end{bmatrix}$$

$$A = \begin{bmatrix} 3 & 2 \\ 5 & -1 \end{bmatrix} \quad B = \begin{bmatrix} 6 & 4 & -2 \\ 0 & 7 & 1 \end{bmatrix}$$

$$C = \begin{bmatrix} 3 \cdot 6 + 2 \cdot 0 & 3 \cdot 4 + 2 \cdot 7 & 3 \cdot (-2) + 2 \cdot 1 \\ & & \\ & & \end{bmatrix}$$

$$C = \begin{bmatrix} 18 & 26 & -4 \\ & & \\ & & \end{bmatrix}$$

$$A = \begin{bmatrix} 3 & 2 \\ 5 & -1 \end{bmatrix} \quad B = \begin{bmatrix} 6 & 4 & -2 \\ 0 & 7 & 1 \end{bmatrix}$$

$$C = \begin{bmatrix} 3 \cdot 6 + 2 \cdot 0 & 3 \cdot 4 + 2 \cdot 7 & 3 \cdot (-2) + 2 \cdot 1 \\ 5 \cdot 6 + (-1) \cdot 0 & 5 \cdot 4 + (-1) \cdot 7 & 5 \cdot (-2) + (-1) \cdot 1 \\ & & \end{bmatrix}$$

$$C = \begin{bmatrix} 18 & 26 & -4 \\ 30 & 13 & -11 \end{bmatrix}$$

Matrizes

```
main.c
1  #include <stdio.h>
2
3  int main() {
4      // Matrizes de entrada
5      int A[2][2] = { {3, 2}, {5, -1} };
6      int B[2][3] = { {6, 4, -2}, {0, 7, 1} };
7
8      // Matriz resultado C (2x3)
9      int C[2][3];
10
11     for (int i = 0; i < 2; i++) {          // linhas de A
12         for (int j = 0; j < 3; j++) {      // colunas de B
13             C[i][j] = 0;
14             for (int k = 0; k < 2; k++) C[i][j] += A[i][k] * B[k][j];
15         }
16     }
17
18     // Imprimindo a matriz resultado C
19     printf("Resultado da multiplicação A x B (matriz C):\n");
20     for (int i = 0; i < 2; i++) {
21         for (int j = 0; j < 3; j++) printf("%d ", C[i][j]);
22         printf("\n");
23     }
24
25     return 0;
26 }
```

Resultado da multiplicação A x B (matriz C):

```
18 26 -4
30 13 -11
```

```
#include <stdio.h>

int main() {
    // Matrizes de entrada
    int A[2][2] = { {3, 2}, {5, -1} };
    int B[2][3] = { {6, 4, -2}, {0, 7, 1} };

    // Matriz resultado C (2x3)
    int C[2][3];

    for (int i = 0; i < 2; i++) {          // linhas de A
        for (int j = 0; j < 3; j++) {      // colunas de B
            C[i][j] = 0;
            for (int k = 0; k < 2; k++) C[i][j] += A[i][k] * B[k][j];
        }
    }

    // Imprimindo a matriz resultado C
    printf("Resultado da multiplicação A x B (matriz C):\n");
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 3; j++) printf("%d ", C[i][j]);
        printf("\n");
    }

    return 0;
}
```